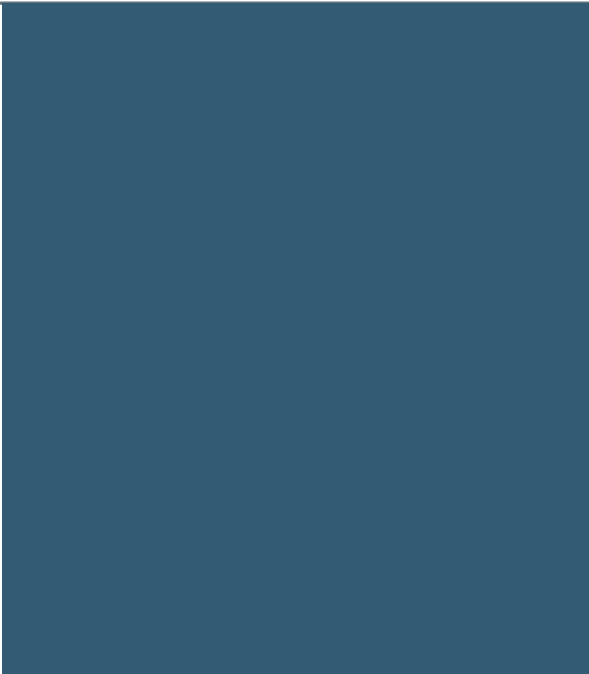




Cahier des Charges Projet ODOS

Projet 2025-2026

MANUEL BASSIEN



1. CONTEXTE & OBJECTIFS

Contexte voire annexe pour photo du projet :

- Projet local : `c:\Users\MANUEL\AppData\Local\VSCode\project`
- Client mobile : React Native avec Expo + expo-router.
- Stack client actuelle : Expo, expo-router, Supabase (auth + BDD), TensorFlow.js (fichiers `ml/index.ts`, `ml/recommendationModel.ts`).
- Fichiers importants côté client : `app/_layout.tsx`, `app/(tabs)/login.tsx`, `app/(tabs)/interests.tsx`, `app/(tabs)/database/supabaseclient.ts`, `app/context/AuthContext.tsx`, `app/context/interestcontext.tsx`, `app/data/activities.ts`, `app/components/RecommendedActivities.tsx`, etc.
- `.env` contient : `EXPO_SUPABASE_URL`, `EXPO_SUPABASE_ANON_KEY` — clé actuellement hardcodée (sécurité à corriger).
- Problèmes observés : warnings expo-router (fichiers non-page dans `app/`), `GetEnv.NoBoolean` (variable bool mal typée), erreurs réseau `Network request failed`, bundling warnings, secrets exposés.
- **Objectif majeur** : conserver client Expo, ajouter / remplacer Supabase par backend Symfony (API REST) pour gérer auth, activités, intérêts, recommandations, profils, favoris, notation, commentaires, etc. Préserver BDD PostgreSQL (poss. migrer depuis Supabase).

Objectif du cahier des charges : produire un plan d'implémentation complet, sécurisé et testable pour intégrer un Backend Symfony et migrer la logique/BDD existante, tout en corrigeant problèmes côté client et finir le projet dans un temps définis qui est l'année scolaire 2025-2026.

2. PORTEE DU PROJET

Inclus

- Conception et développement du backend **Symfony** (API REST)
- Migration des données PostgreSQL hébergées sur Supabase vers l'instance PostgreSQL utilisée par Symfony *ou* maintien de Supabase comme DB hébergée.
- Adaptation du client React Native Expo : suppression des mauvaises pratiques (clés en dur), redirection des appels vers API Symfony, correction warnings expo-router, intégration auth JWT, gestion erreurs réseau.
- Mise en place CI/CD
- Tests unitaires & d'intégration.
- Documentation.
- Mise en place monitoring de logs structuré.

3. DESCRIPTION GENERALE

ODOS est une Application mobile Touristique de découverte d'activités personnalisées avec aide de machine learning. Les utilisateurs s'inscrivent, sélectionnent leurs centres d'intérêt, reçoivent les recommandations, peuvent marquer favoris, noter, commenter, rechercher et filtrer les activités. Le Modèle entraîné par les choix de l'utilisateur deviendra avec le temps de plus en plus précis dans ses recommandations.

3.2 Utilisateurs cibles

- **Utilisateur final (USER)** : s'inscrit, gère profil, intérêts, consulte activités, commente, note et reçoit recommandations.
- **Administrateur (ADMIN)** : gestion CRUD activités, catégories, modération commentaires, accès reporting.
- **Service ML / Ops** : consomme logs pour entraîner modèles.

3.3 Les Priorités

MUST

1. Authentification sécurisée (signup/login/reset) — JWT + refresh.
2. CRUD activités & catégories pour admin.
3. Utilisateur : page principale + intérêts + profil.
4. Recommandations basiques (TF.js côté client existant — prendre en charge).
5. Favoris, notes, commentaires.
6. API RESTful
7. Migration des données existantes Supabase → nouvelle DB.
8. Suppression des clés en dur côté client, gestion via env/secrets Expo.

SHOULD

1. Endpoint d'import/export activités (CSV).
2. Endpoint pour logs de recommandation (tracking).
3. Dashboard admin basique (statistiques d'usage).

COULD

1. Service recommandations côté serveur (microservice Python).
2. Notifications push (Expo Push).
3. Analytics & A/B testing.

3.4 Parcours utilisateurs (exemples)

1. **Onboarding** : installation → connexion → sélection intérêts → recommandations liste.
2. **Exploration** : recherche → filtre par catégorie → voir détail activité → noter/commenter → ajouter aux favoris.

3. **Admin** : login admin → accéder dashboard → CRUD activité → modération commentaire.

4. SPECIFICATION TECHNIQUE & ARCHITECTURE

4.1 Principes techniques

- Backend: **Symfony 6.x** (API Platform optionnel) ou microservices si évolutivité majeure.
- Auth
- BDD: **PostgreSQL** (compatible Supabase). Doctrine ORM + migrations.
- Storage média: Supabase Storage possible temporairement.
- ML: rester sur TF.js client pour MVP; prévoir API pour recommandations serveur (REST).
- CI/CD: GitHub Actions
- Infrastructure: Docker compose pour dev; hébergement prod
- Observabilité
- Secrets: stockés en variables d'environnement via provider secrets

4.2 Diagramme d'architecture (logique)

- Client Expo (iOS/Android)
 - Auth local -> requests Bearer -> API Symfony
 - TF.js local (optional) -> calls internal recommendation endpoint for sync tracking
- API Symfony (HTTPS)
 - Endpoints REST pour auth, users, activities, interests, recommendations, favorites, ratings, comments, admin.
 - Services
- BDD PostgreSQL
- Object Storage
- Services externe: Supabase

5. AUTHENTIFICATION & SECURITE

5.1 Protections applicatives

- Validation & sanitation stricte (Symfony validators + DTOs). Rejeter tout payload non attendu.
- Protection contre injection SQL (Doctrine + param binding)
- Rate limiting: config par endpoint (ex: login limité).
- CORS : whitelist des domaines
- HTTPS
- Politique de Mot de Passe
- Stockage secret: pas dans code ni .env dans repo

5.2 Gestion des clés & rotation

- .env.example avec placeholders
- Recommandation : rotation trimestrielle pour clés sensibles
- Ne jamais hardcoder EXPO_SUPABASE_ANON_KEY dans code client.

5.3 Conformité & RGPD

- Minimiser stockage données personnelles.
- Politique de Confidentialité (consentement explicite).
- Droits utilisateur : endpoint pour export et delete.

6. MIGRATION DEPUIS SUPABASE

1. Les Options

1. **Garder Supabase comme Postgres hébergé** et connecter Symfony dessus. (Faible friction)
 - Avantages : rapide, pas d'export/import, Supabase Storage et Auth peuvent rester pour transition.
 - Inconvénients : dépendance vis-à-vis d'un service externe, contrôle limité sur infra.
2. **Exporter les données Supabase (Postgres) et importer dans nouvel hébergement PostgreSQL** (auto-hébergé ou cloud).
 - Avantages : contrôle total, consolidation, meilleure intégration.
 - Inconvénients : effort d'export/import, downtime possible.

2. Plan export/import

Points clés

- Vérifier extensions Postgres requises
- Mapping des users/passwords: Supabase Auth qui était utilisé possède son propre système de table et d'authentification si on utilise Symfony Auth, migrer uniquement user emails et créer hash mots de passe (ou demander reset).
- Media: exporter le stockage (Supabase Storage) vers alternative
- TF.js: fichiers ML à conserver surement.

7. ML / RECOMMANDATIONS

Recommander du côté serveur

Architecture

- Microservice dédié (Symfony + PHP ML wrapper ou Python Flask/FastAPI).
- Récupère les infos : user profile / interest ID / context return donc la liste.
- Log pour entraîner hors ligne.

8. UI/UX ET MAQUETTES

Pages clés (wireframes)

- Onboarding
- Auth : Login / Register / Password Reset
- Onboarding intérêts (checkbox grid)
- Liste activités (le feed) avec filtres / recherche
- Détail activité (images + map + actions: favorite, comment, rate)
- Profil / Paramètres
- (Admin : CRUD activités / modération)

Recommandations composants React Native

- L'UI: **React Native** et **React Native Elements**
- Navigation : `expo-router` + stack/tabs organised.
- State: React Context for auth + Redux ou autre
- Image: react-native-fast-image (ou Expo equivalent)

9. RISQUES & MITIGATIONS

Risque	Probabilité	Impact	Mitigation
Secrets exposés (actuel)	Élevée	Élevé	Rotation immédiate, utiliser secret manager, audit, remplacer clés hardcodées
Migration DB lourde / perte de données	Moyenne	Élevé	Tests sur staging, backups, plan rollback, fenêtre maintenance
Modèle ML trop lourd pour mobiles	Moyenne	Moyen	Option serveur
Dépendance Supabase non souhaitée	Moyenne	Moyen	Plan B : exporter DB vers infra contrôlée
Latence API	Moyenne	Moyen	Profiling, caching (Redis), optimisation queries
Coût infra supérieur aux prévisions	Moyenne	Moyen	Choix infra ajusté, monitoring coûts

10. MODIFICATIONS : RESUME DE L'EVOLUTION

1. Sécuriser secrets

- Supprimer EXPO_SUPABASE_ANON_KEY hardcodée, utiliser .env + expo secrets (EAS secrets) / runtime config.
- Ajouter .env.example (voir annexe).

2. Routes non-pages

- Déplacer utilitaires / composants non-page hors de app/ (par ex src/components/ ou app/_internal/) ou marquer export default comme null selon expo-router recommendations.

3. Gestion GetEnv.

- Corriger types d'env (ex: EXPO_FEATURE_ENABLED=true)

4. Network request failed

- Vérifier base URL, CORS, HTTPS

5. Adaptations API

- Remplacer appels Supabase par appels REST (axios/fetch).
Implémentation centralisé utilisant Access token + refresh flow.

6. State & caching

- Ajouter React Query for API caching + invalidation.

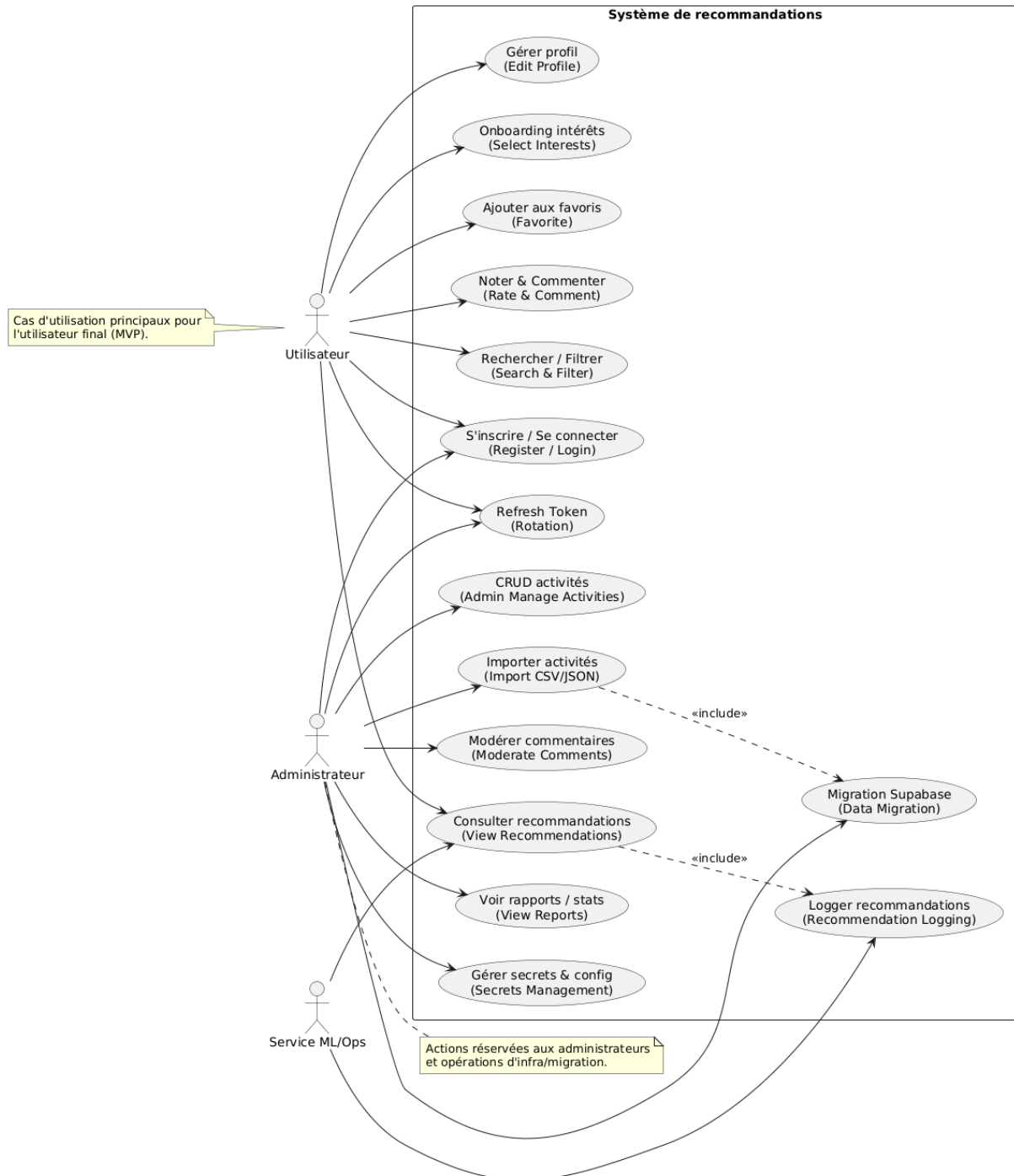
7. TF.js

- Changer vers un hébergement serveur pour les recommandations

ANNEXES :

Schéma UML:

Cas d'utilisation et de Classe :



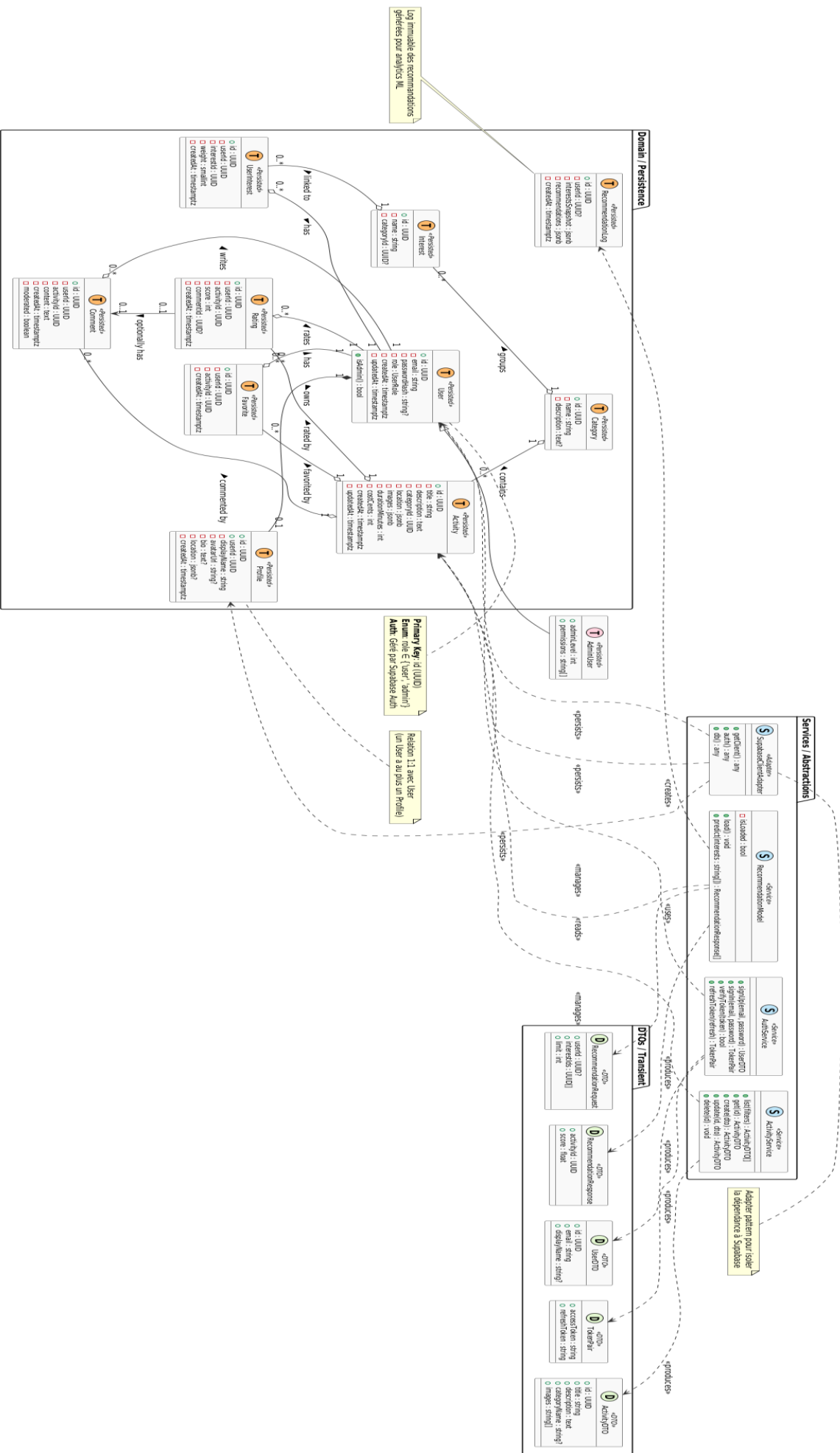
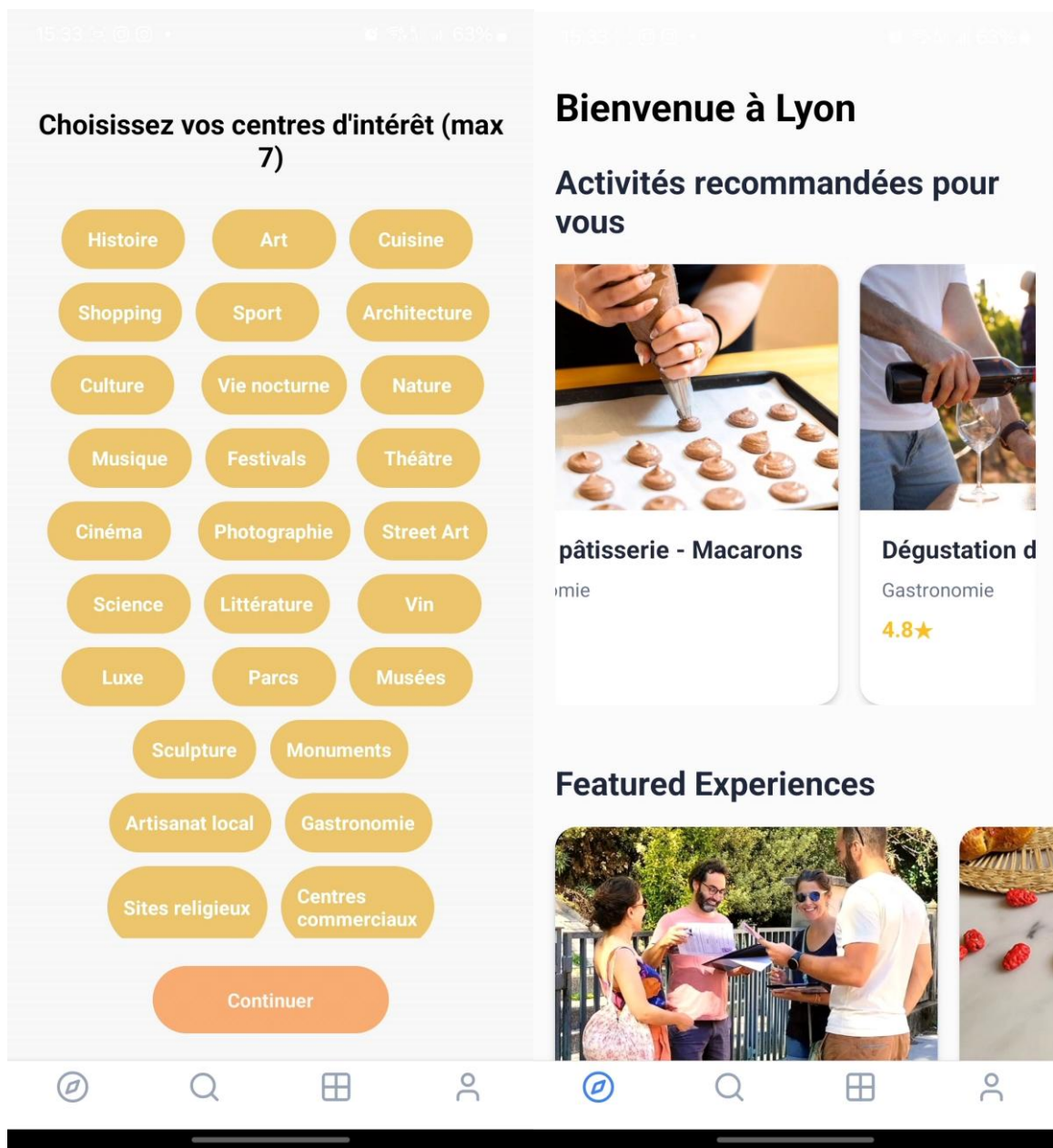
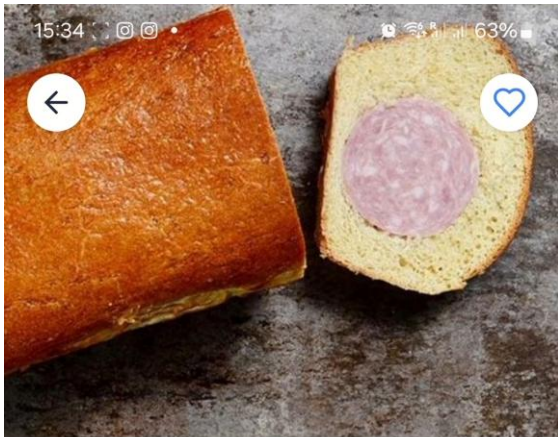


Image de l'application à ce jour :





Cours de cuisine bouchon lyonnais

★ 4.6 (68 reviews)

🕒 3 heures

📍 Rue Mercière, 69002 Lyon

\$65 per person

A propos de cette expérience

Participez à un atelier culinaire pour apprendre à cuisiner les classiques lyonnais comme la quenelle ou l'andouillette.

Réserver



test2@gmail.com

⚙ Settings

♥ Favorites

✍ Changer mes intérêts

🕒 History

[→] Déconnexion

